

Project Report

Submitted by:

Dipanshi Gaur

Project: Secure Login System with User Details & File Access

1. Introduction

Authentication and authorization are important parts of almost every modern application. A system should not only verify whether a user is registered, but should also make sure that the user can access only the information and files that belong to them.

The objective of this project was to develop a secure login system with registration, login, logout, user profile access and user-specific file access. The same requirements were implemented using two different approaches.

The first implementation uses a **custom backend built with Node.js, Express.js and PostgreSQL**. The second implementation uses **Appwrite as a managed backend service**.

The provided index.html was used as the testing client for both implementations. No separate GUI was developed because the main focus of the task was authentication, authorization, session management and data isolation rather than UI design.

The project was also tested with three separate users — Alice, Bob and Carol — to verify that one user cannot access another user's profile or files.

2. Objectives

The main objectives of the project were:

- Implement user registration using email and password.
- Implement secure login and session management.
- Implement logout with server-side session invalidation.
- Provide a protected /me route for the logged-in user's profile.
- Provide a protected /files route for listing the user's files.
- Provide access to individual files through /files/:id.
- Allow users to download their own files.
- Prevent users from accessing files belonging to other users.
- Store passwords securely using hashing.
- Add protection against repeated failed login attempts.
- Implement the same functionality using Appwrite.
- Test the system with multiple users.
- Document the security decisions and implementation approach.

3. Technologies Used

Custom Backend

- **Node.js** - Backend runtime environment.
- **Express.js** - Used to create REST APIs.

- **PostgreSQL** - Used for storing users, sessions and file metadata.
- **bcrypt** - Used for securely hashing passwords.
- **Express Rate Limit** - Used to limit repeated failed login attempts.
- **HTML and JavaScript** - Used for the testing interface.

Appwrite

- **Appwrite Authentication** - Used for registration and login.
- **Appwrite Database** - Used for storing file metadata.
- **Appwrite Storage** - Used for file storage.
- **Appwrite Web SDK** - Used to connect the frontend with Appwrite.

4. System Implementation

The project contains two implementations of the same basic functionality.

4.1 Custom Backend

The custom backend provides the following APIs:

POST /register

POST /login

POST /logout

GET /me

GET /files

GET /files/:id

GET /files/:id/download

During registration, the user's password is hashed using bcrypt before it is stored in PostgreSQL.

During login, the entered password is compared with the stored password hash. After successful authentication, a random session token is generated and stored securely on the server side. The browser receives the session through an HTTP-only cookie.

The /me endpoint returns information about the currently logged-in user. The /files endpoint returns only the files belonging to that user.

For individual file access, the backend checks both the existence of the file and its ownership. A user can access or download a file only when it belongs to their account.

4.2 Appwrite Implementation

The second implementation uses Appwrite as a managed backend.

Appwrite Authentication is used for creating accounts, login, session management and logout.

A database collection is used to store file information such as:

- Owner ID
- File name
- MIME type
- File size
- Storage file ID

Appwrite Storage is used for the actual files.

User-specific permissions are configured for database documents and storage files. This ensures that users can access only the files for which they have permission.

For example, Alice can access Alice's files, while Bob cannot access Alice's files.

5. Authentication and Security

Security was an important part of the implementation.

Passwords are never stored as plain text. The custom backend uses bcrypt to hash passwords before storing them in PostgreSQL.

For the custom backend, I used server-side cookie sessions instead of JWTs. This makes session revocation straightforward during logout because the session can be marked as revoked in the database.

The session cookie is configured as HTTP-only and uses SameSite protection. Sessions also have an expiry time.

The login endpoint uses rate limiting to restrict repeated failed login attempts.

The system also uses a generic error message for invalid login credentials so that it does not directly reveal whether a particular email address exists.

SQL queries are parameterized to reduce the risk of SQL injection.

For file access, ownership is checked before returning file information or allowing a download.

In the Appwrite implementation, access control is handled through Appwrite's document and file permissions.

6. User and File Access

Three development users were used during testing:

User	Email
Alice	alice@example.com
Bob	bob@example.com
Carol	carol@example.com

Each user has their own files.

The system was tested to make sure that:

- Alice can access Alice's files.
- Bob can access Bob's files.
- Carol can access Carol's files.
- Alice cannot access Bob's files.
- Bob cannot access Alice's files.
- Carol cannot access Alice's or Bob's files.
- Logged-out users cannot access protected resources.

This confirmed that authentication and authorization were working separately and that file ownership was being properly enforced.

7. Testing

The following tests were performed on the application:

Test	Expected Result
New user registration	Successful
Valid login	Successful
Wrong password	Login rejected
Wrong email	Login rejected
Get current user	User details returned
Get user's files	Only user's files returned
Access own file	Allowed
Download own file	Allowed
Access another user's file	Denied
Download another user's file	Denied
Non-existent file	404 response
Logout	Session invalidated
Access after logout	401 response
Repeated failed login	Rate limited
Appwrite user isolation	Verified
Testing was carried out for both the custom backend and Appwrite implementation.	

8. Challenges and Design Decisions

One of the main challenges was implementing proper file-level authorization. It was important to make sure that being

logged in did not automatically give a user access to every file.

For the custom backend, the authenticated user's ID is obtained from the session instead of accepting a user ID directly from the client.

For Appwrite, the required document and storage permissions had to be configured for individual users.

Another important design decision was using server-side sessions in the custom backend. This made logout and session revocation easier to manage.

9. Running the Project

For the custom backend, the following commands are used:

```
cd custom-backend
```

```
npm install
```

```
npm run migrate
```

```
npm run seed
```

```
npm run dev
```

The backend runs on port 3000.

The testing interface can be served on port 8080 using:

```
npx http-server . -p 8080
```

The application can then be accessed through the local browser.

10. Conclusion

The project successfully implements a secure login and file-access system using both a custom backend and Appwrite.

The custom implementation demonstrates authentication, password hashing, session management, database operations and authorization using Node.js, Express.js and PostgreSQL.

The Appwrite implementation demonstrates the use of a managed backend for authentication, database and storage while maintaining user-specific permissions.

Testing with Alice, Bob and Carol confirmed that users can access their own information and files while unauthorized access to other users' resources is prevented.

Overall, the project helped in understanding practical concepts of authentication, authorization, session management, database security and user data isolation.